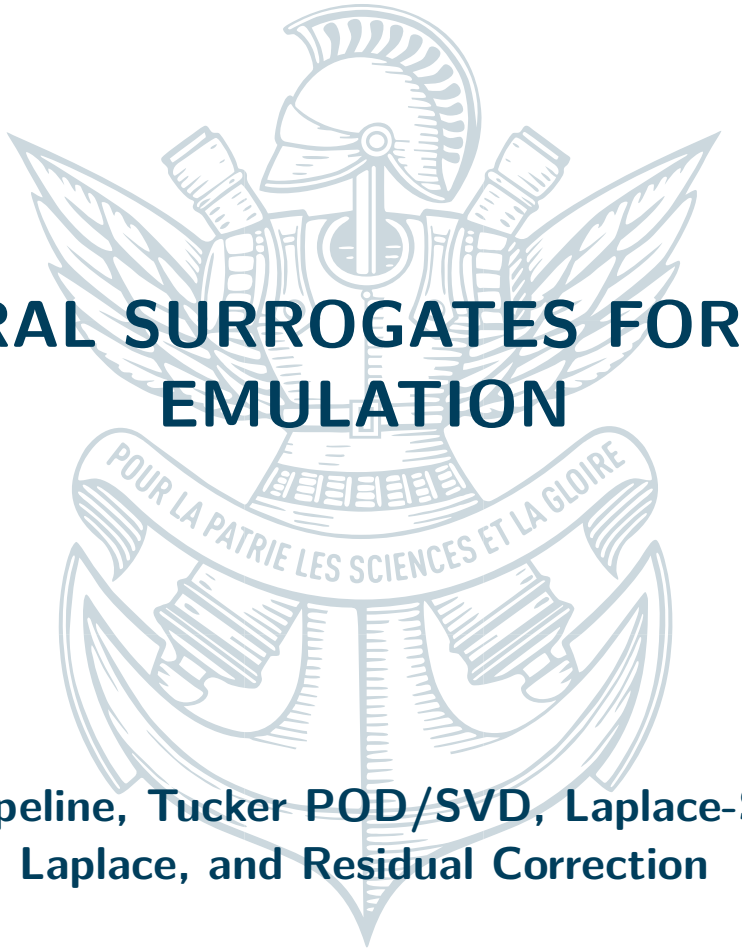


NEURAL SURROGATES FOR PDE EMULATION

Stationary pipeline, Tucker POD/SVD, Laplace-SVD, Latent Laplace, and Residual Correction

April 14, 2026

Keyvan Attarian

CONTENTS

1 INTRODUCTION

Numerical simulation of physical systems governed by partial differential equations (PDEs) is indispensable in science and engineering, yet often prohibitively expensive for applications requiring repeated solver evaluations over large parameter spaces—such as design optimisation, uncertainty quantification, or real-time data assimilation. Neural surrogates address this bottleneck by learning a fast, differentiable approximation of the map $\boldsymbol{\theta} \mapsto U$ from physical parameters to solution fields, replacing minute-long solver calls with millisecond-scale inference.

This report documents the successive surrogate architectures developed throughout this project. Two physical settings are considered. The first is a stationary 2-D convection-diffusion problem, which serves as a controlled testbed for variational autoencoder (VAE)-based approaches. The second, and principal, problem concerns transient CH4 concentration fields, for which a hierarchy of increasingly expressive methods is presented: from Tucker POD decomposition and SVD-based spectral representations, through a frequency-conditioned autoencoder latent space, to a full end-to-end latent surrogate with residual correction post-processing.

2 PRELIMINARY STUDY: STATIONARY DATASET

2.1 Problem Statement

We consider the stationary convection-diffusion equation on the unit square $\Omega = [0,1]^2$:

$$-D\,\Delta U + \boldsymbol{b}\cdot\nabla U = f \quad \text{in } \Omega, \qquad U = 0 \quad \text{on } \partial\Omega, \tag{1}$$

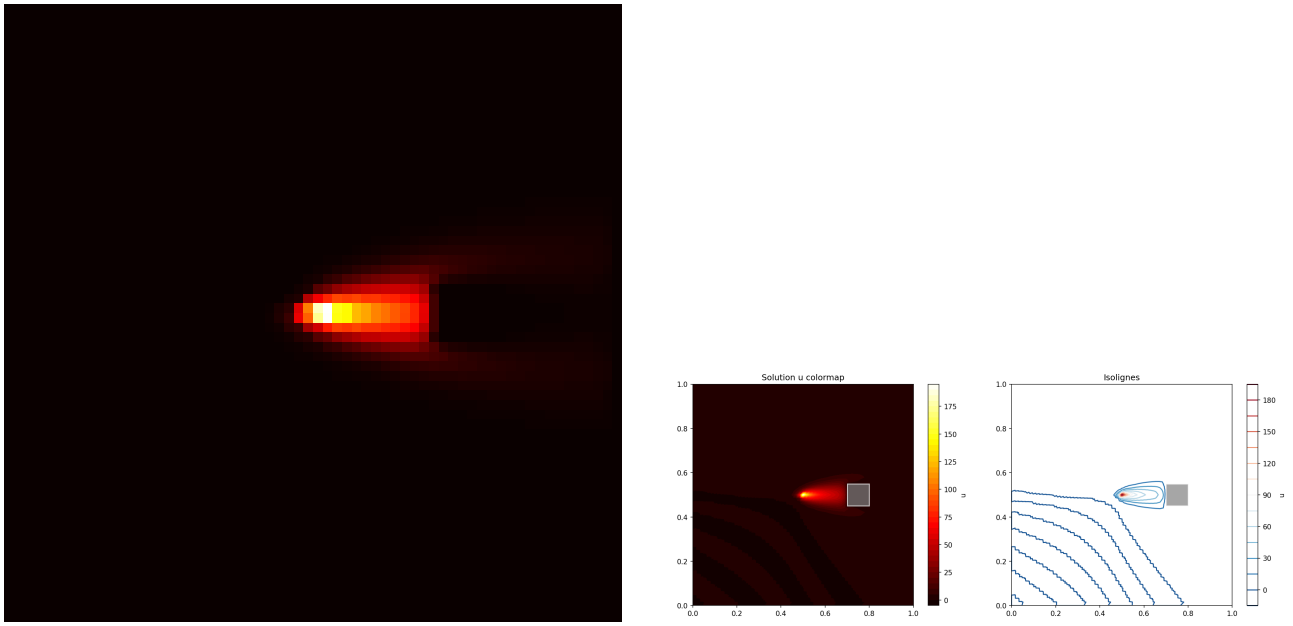
where $D > 0$ is the scalar diffusivity, $\boldsymbol{b} = (b_x, b_y)$ the convection velocity, and f the intensity of a point source. The parameter vector is $\boldsymbol{\theta} = (D, b_x, b_y, f) \in \mathbb{R}^4$, and the goal is to learn the map $\boldsymbol{\theta} \mapsto U \in \mathbb{R}^{N\times N}$ without calling the finite element solver at test time.

2.2 Dataset Generation

The reference solver (`utils/sim.py`) discretises (1) with P1 finite elements on a triangular mesh using FEniCS/DOLFINx. The convective term is stabilised via SUPG/GLS with a locally computed parameter τ. The finite element solution is then interpolated onto a uniform $N \times N$ Cartesian grid. The dataset generator (`utils/dataset_generator.py`) samples $\boldsymbol{\theta}$ uniformly at random within prescribed physical ranges and stores the resulting pairs $(\boldsymbol{\theta}, U)$ in `dataset/dataset.npz`.

Normalisation. Each spatial component of U is centred and linearly rescaled to $[-1, 1]$ using per-pixel statistics computed on the training split. The parameter vector $\boldsymbol{\theta}$ is standardised with pre-computed mean and standard deviation. All statistics are serialised into the checkpoint.

2.3 Two-Stage Architecture



(a) Grid of solution fields for varying $\boldsymbol{\theta}$.

(b) Example stationary solution U on $[0,1]^2$.

Figure 1: Stationary convection-diffusion dataset.

2.3.1 • Stage 1 — Variational Autoencoder

A VAE compresses solution fields $U \in \mathbb{R}^{1\times N\times N}$ into a low-dimensional latent code $z \in \mathbb{R}^{d_z}$ (default $d_z = 32$). The encoder is a four-block strided convolutional network followed by two linear heads producing the posterior mean μ and log-variance $\log \sigma^2$. The decoder maps a sampled code back through a linear projection and four ConvTranspose2d+BatchNorm+ReLU blocks to the image space.

Training minimises the negative evidence lower bound augmented with a spatial gradient penalty:

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q(z|U)}\left[\|U - \hat{U}\|^2 + \lambda_\nabla \|\nabla U - \nabla \hat{U}\|^2\right] + \beta\,\mathrm{KL}\big(q(z|U)\,\|\,\mathcal{N}(0, I)\big), \tag{2}$$

where a *free-bits* threshold per latent dimension prevents premature collapse of the KL term.

2.3.2 • Stage 2 — Indirect Decoder $\boldsymbol{\theta} \to z \to U$

With the VAE decoder frozen, an `IndirectDecoder` network learns the regression $\boldsymbol{\theta} \to z$ with a two-layer MLP (hidden size 128, ReLU). The predicted code is then passed through the frozen decoder. The loss is MSE on the reconstructed field plus the same spatial gradient term as in (2). This two-stage strategy decouples representation learning from the parameter-to-latent regression, resulting in more stable training.

2.4 Interactive Application

A Streamlit application (`stationary/app.py`) wraps inference into an interactive interface: sliders expose D, the magnitude and angle of $\boldsymbol{b}$, and f, and the predicted field is optionally compared against the nearest dataset neighbour. Model and dataset loading are cached for real-time response.

2.5 Results

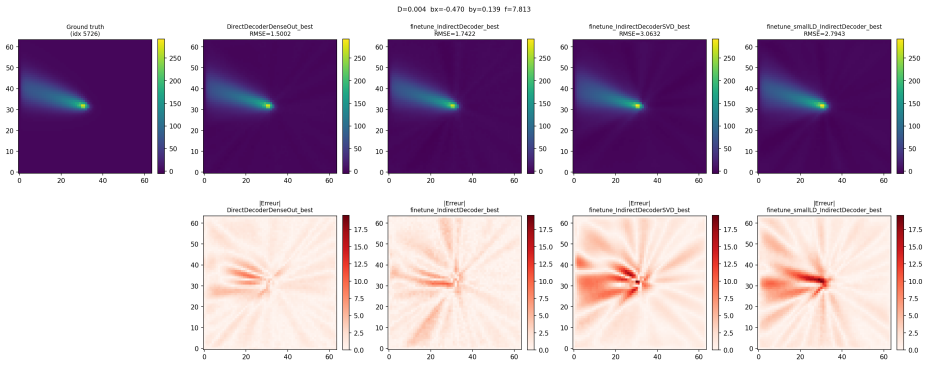


Figure 2: Streamlit interface: surrogate prediction (left) vs. nearest dataset sample (right).

3 TRANSIENT CH4 CONCENTRATION FIELDS

3.1 Dataset and Problem Formulation

3.1.1 • Dataset

The primary dataset (`ch4_rotated`) contains approximately 8 100 simulations of transient CH4 concentration fields. Each simulation yields $N_t = 150$ time steps on a 200×200 spatial grid, corresponding to a raw tensor of shape $(8100, 150, 200, 200)$. Physical parameters are $\boldsymbol{\theta} = (k, A, C)$ (reaction rate, amplitude, initial concentration) stored in `doe_rotated.npy`. Data are accessed in memory-map mode to avoid materialising the full $\sim$17 GB dataset in RAM.

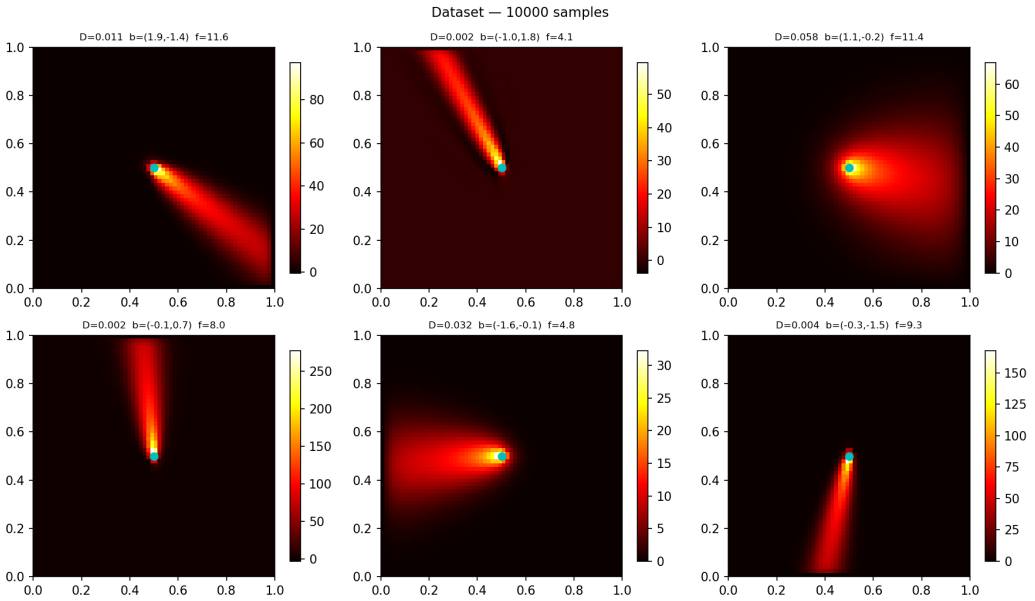


Figure 3: Dataset preview: representative CH4 concentration fields at selected time steps.

3.1.2 • Temporal Representation via the Laplace Transform

A key modelling choice for the transient problem is to operate in the *Laplace domain* rather than directly in the time domain. The numerical Laplace transform is computed along the Bromwich contour:

$$\hat{U}(s_k) = \int_0^T U(t)\, e^{-s_k t}\, \mathrm{d}t \approx \mathrm{DFT}\big[U(t) \cdot w_t \cdot e^{-\gamma t}\big], \qquad s_k = \gamma + i\omega_k, \tag{3}$$

where w_t are trapezoidal quadrature weights and $\gamma \geq 0$ is a damping parameter. Inversion is performed by IFFT with a compensating exponential factor. By the Hermitian symmetry of real-valued signals, only the $N_{\text{half}} = N_t/2 + 1$ non-redundant frequencies need to be modelled; the full spectrum is reconstructed via $\hat{U}(s_{N_t-k}) = \overline{\hat{U}(s_k)}$.

This representation decomposes the temporal dynamics into independent frequency components, enabling the construction of specialised surrogates per frequency and facilitating both latent space learning and residual correction.

3.2 Latent Space Representation

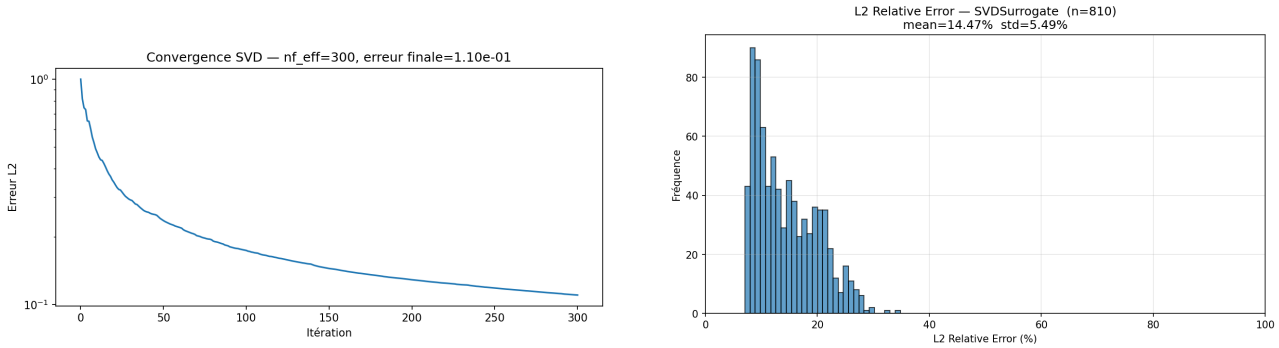
A central challenge in the transient setting is to find a compact, learned representation of the Laplace coefficient fields $\hat{U}(s_k) \in \mathbb{C}^{N\times N}$ (equivalently, the pair $(\Re(\hat{U}_k), \Im(\hat{U}_k)) \in \mathbb{R}^{2\times N\times N}$). Two families of representations are explored.

3.2.1 • SVD-Based Approaches

Tucker POD decomposition. Following the hybrid twin framework of [**?**], the Tucker SVD (`utils/SVD_Amine_3D.py`) factorises the spatially sub-sampled training tensor $\boldsymbol{H} \in \mathbb{R}^{n_r\times n_s\times N_t}$ as

$$H_{r,s,t} \approx \sum_{j=1}^{n_f} \alpha_j\, F_{r,j}\, G_{s,j}\, P_{t,j}, \tag{4}$$

with spatial basis $\boldsymbol{F}$, temporal basis $\boldsymbol{P}$, singular values $\boldsymbol{\alpha}$, and simulation-specific coefficients $\boldsymbol{G}$. Once the bases are fixed, the problem reduces to learning $\boldsymbol{\theta} \mapsto \boldsymbol{g} \in \mathbb{R}^{n_f}$ with a simple MLP. This approach is extremely lightweight but fundamentally limited by the linear structure of (4).



(a) Reconstruction error vs. number of retained modes n_f.

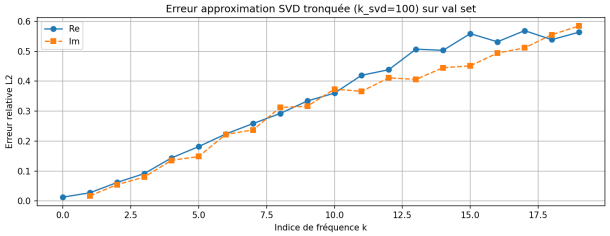
(b) Relative L^2 error on the test set (Tucker POD).

Figure 4: Tucker POD surrogate.

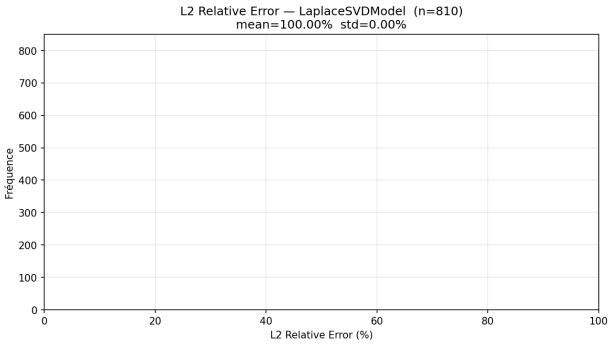
Per-frequency SVD in the Laplace domain. For each Laplace frequency k, a truncated SVD is computed on the matrix of training coefficients (real and imaginary parts independently):

$$\Re(\hat{\boldsymbol{U}}_k) \approx \boldsymbol{c}^{(k,r)} \big(\boldsymbol{V}^{(k,r)}\big)^\top, \quad \Im(\hat{\boldsymbol{U}}_k) \approx \boldsymbol{c}^{(k,i)} \big(\boldsymbol{V}^{(k,i)}\big)^\top,$$

where $\boldsymbol{V} \in \mathbb{R}^{N^2 \times k_{\text{svd}}}$ and $\boldsymbol{c} \in \mathbb{R}^{k_{\text{svd}}}$ are the leading right singular vectors and associated coefficients respectively. A lightweight three-layer MLP per frequency then predicts the normalised coefficients $\hat{\boldsymbol{c}}$. This approach, implemented as `LaplaceSVDModel`, combines the Laplace decomposition with a linear spatial basis, enabling frequency-specific compression.



(a) SVD reconstruction error vs. number of components ($k_{\text{svd}} = 100$).



(b) Relative L^2 error on the test set (Laplace SVD).

Figure 5: SVD surrogate in the Laplace domain.

Limitations of SVD-based representations. Both SVD approaches impose a *linear* structure on the latent space: the field $\hat{U}_k$ is constrained to lie in a fixed linear subspace spanned by pre-computed basis vectors. While this is computationally simple and theoretically well-founded for low-rank fields, it becomes a bottleneck when the physical fields exhibit complex, non-linear variation across the parameter space. In particular, the spatial structures of $\hat{U}_k$ can vary non-linearly with $\boldsymbol{\theta}$, making a fixed linear basis suboptimal.

3.2.2 • Autoencoder-Based Latent Space

To overcome the linearity constraint, we learn a non-linear latent space via a frequency-conditioned autoencoder (`LaplaceAE`, `models/laplace_ae_surrogate.py`).

Sinusoidal frequency conditioning. Since a single autoencoder must handle Laplace fields at all frequencies simultaneously, it is conditioned on the normalised frequency ratio $f = k/N_{\text{half}} \in [0, 1]$. A NeRF-style sinusoidal positional encoding provides a dense, multi-resolution representation of this scalar:

$$\boldsymbol{e}(f) = \text{MLP}\Big(\big[\sin(2^i \pi f),\ \cos(2^i \pi f)\big]_{i=0}^{L-1}\Big) \in \mathbb{R}^{64}, \tag{5}$$

with $L = 8$ frequency levels. This encoding allows the network to discriminate between neighbouring Laplace frequencies while capturing coarse spectral structure through the lower-frequency components.

Architecture. The FiLM (Feature-wise Linear Modulation) mechanism [1] injects the frequency embedding at each network layer by computing affine modulation parameters $(\gamma_\ell, \beta_\ell)$ from $\boldsymbol{e}(f)$:

$$\boldsymbol{x} \leftarrow \boldsymbol{x} \odot (1 + \gamma_\ell) + \beta_\ell. \tag{6}$$

The encoder (`LaplaceEncoder`) applies three strided Conv2d blocks ($2 \to 16 \to 32 \to 64$ channels, stride 2), each followed by FiLM conditioning, then maps the spatial features to $z \in \mathbb{R}^{d_z}$ via two linear layers. The scaling

parameters γ_ℓ are passed through tanh to avoid numerical instability in half-precision training. The decoder (`LaplaceDecoder`) reverses this with three ConvTranspose2d+BatchNorm+ReLU blocks, FiLM-modulated at each level, followed by two independent refinement branches (one for $\Re$, one for $\Im$) that produce the final two-channel output.

The training objective combines reconstruction MSE and a ridge regularisation on the decoder outputs:

$$\mathcal{L}_{\text{AE}} = \|\hat{U} - U\|^2 + \beta \|\hat{U}\|^2. \tag{7}$$

The training dataset is flattened over the (simulation, frequency) dimensions and reshuffled between epochs to prevent frequency-specific overfitting.

Advantage over SVD. The autoencoder latent space is non-linear, adaptive, and jointly trained across all frequencies. The shared encoder and decoder weights are specialised to each frequency only through the lightweight FiLM modulation, enabling efficient parameter sharing without sacrificing frequency-specific expressiveness. Empirically, this leads to lower reconstruction error than SVD at comparable latent dimensionality d_z.

3.3 Surrogate and Fine-Tuning

3.3.1 • Per-Frequency Surrogate $\boldsymbol{\theta} \to z$

With the `LaplaceAE` trained and its decoder frozen, we train one lightweight MLP per Laplace frequency to learn the regression $\boldsymbol{\theta} \to z_k$ (`LaplaceLatentSurrogate`). Each surrogate consists of a four-layer MLP (`proj`) with LayerNorm after each linear transformation:

$$\boldsymbol{\theta}_{\text{norm}} \xrightarrow{d_h/2} \xrightarrow{d_h} \xrightarrow{d_h} d_z, \tag{8}$$

where $d_h = 256$ is the hidden width. The shared decoder is injected by reference (not registered as a submodule), avoiding any weight duplication in GPU memory.

The assembled `LaplaceLatentModel` contains N_{half} such surrogates together with a single frozen `shared_decoder`. At inference, all projections $\{\boldsymbol{\theta} \to z_k\}$ are computed in parallel, then a single batched decoder call produces the entire Laplace spectrum:

$$\{z_k\}_{k=0}^{N_{\text{half}}-1} \xrightarrow{\text{batched}} \text{shared\_decoder} \to \{\hat{U}_k\}_k. \tag{9}$$

The time series is recovered by Hermitian symmetry completion followed by IFFT inversion of the Bromwich integral (3).

3.3.2 • End-to-End Fine-Tuning

After independent per-frequency training of the `proj` networks, the shared decoder is unfrozen and the complete model is fine-tuned end-to-end (`transient/finetune_decoder_laplace.py`). Differential learning rates are applied: the surrogate MLPs are updated with a larger step size $\eta_{\text{surr}} = 5 \times 10^{-5}$, while the decoder uses a smaller rate $\eta_{\text{dec}} = 10^{-5}$, preserving the learned spectral decoder structure while allowing it to adapt to the surrogate outputs. The fine-tuning dataset is reorganised in frequency-first order so that each mini-batch covers all training simulations for a single frequency, enabling a single batched decoder call per gradient step and substantially reducing wall-clock time.

3.3.3 • Results

3.4 Post-Processing and Results

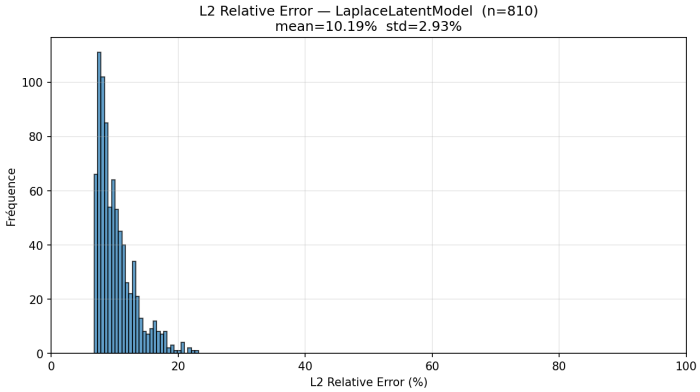


Figure 6: Relative L^2 error on the test set after end-to-end fine-tuning (latent Laplace surrogate).

3.4.1 • Residual Correction

Laplace-based surrogates can exhibit oscillatory artefacts—particularly near sharp transitions and in high-gradient regions—due to the finite number of retained spectral frequencies and residual surrogate approximation error. Rather than retraining the surrogate, a lightweight residual corrector is applied *post hoc* frame by frame, learning only the systematic error pattern.

The `CorrectionAE` (`models/correction_ae.py`) is a three-level UNet. Given a predicted frame $U_{\text{pred}} \in \mathbb{R}^{N\times N}$, it is first normalised per-sample: $\tilde{U} = (U_{\text{pred}} - \mu)/\sigma$. The encoder applies three double-convolution blocks (Conv2d 3×3 + ReLU, $\times2$) with MaxPool2d downsampling, with channel progression $1 \to c \to 2c \to 4c$ ($c = 16$). A bottleneck at resolution $N/8$ uses $8c$ channels. The decoder reverses the encoder with ConvTranspose2d upsampling, concatenating skip connections at each level. The output 1×1 convolution is **zero-initialised**, so the model starts from the identity (zero residual correction) and learns corrections gradually:

$$U_{\text{corr}} = U_{\text{pred}} + \sigma \cdot r\big(\tilde{U}\big)\,, \tag{10}$$

where $r(\cdot)$ denotes the UNet output. With base channel width $c = 16$, the model has approximately 481 000 parameters.

3.4.2 • Training the Corrector

To avoid repeated expensive calls to the base surrogate, all $(U_{\text{pred}}, U_{\text{true}})$ pairs are pre-computed and stored as memory-mapped arrays of shape (n_s, k_t, N, N) on disk ($k_t = 20$ randomly sampled frames per simulation). The surrogate is then freed from GPU memory before corrector training begins.

The training objective is an MSE on the normalised residual augmented by a spatial gradient matching term:

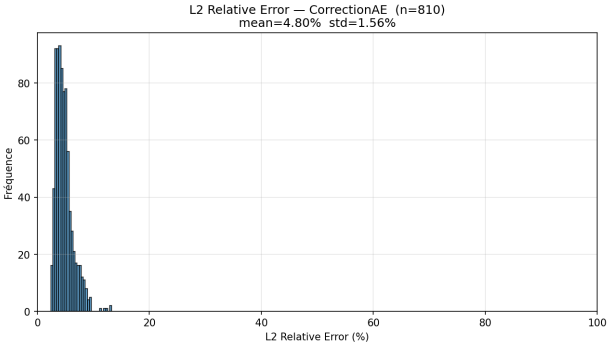
$$\mathcal{L}_{\text{corr}} = \left\| \frac{r_{\text{pred}} - r_{\text{true}}}{\sigma_r} \right\|^2 + \lambda_\nabla \left\| \frac{\nabla r_{\text{pred}} - \nabla r_{\text{true}}}{\sigma_r} \right\|^2, \tag{11}$$

where $r_{\text{pred}} = U_{\text{corr}} - U_{\text{pred}}$, $r_{\text{true}} = U_{\text{true}} - U_{\text{pred}}$, $\sigma_r = \sigma(r_{\text{true}})$, and $\lambda_\nabla = 10$.

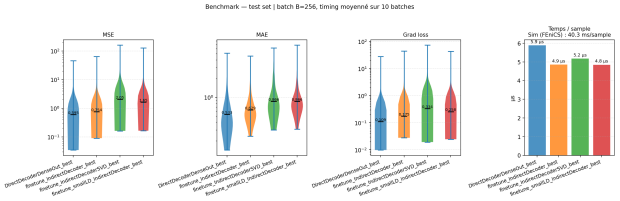
3.4.3 • Full Pipeline

The `CorrectedPipeline` class inherits from `LaplaceLatentModel` and overrides the generation step: after Laplace inversion, all frames pass through the correction UNet in chunks of 64 to manage GPU memory. Surrogate modules are shared by reference with no weight duplication.

3.4.4 • Results and Comparison



(a) Relative L^2 error distribution after residual correction.



(b) Pipeline comparison on the test set.

Figure 7: Residual correction post-processor and overall comparison.

Method	Representation	Parameters	Median L^2_{rel} (%)
Tucker POD	POD coefficients $\boldsymbol{g}$	$\sim$1 M	—
Laplace SVD	SVD coefficients per frequency	—	—
Latent Laplace	Shared autoencoder latent code	—	—
Latent Laplace + AEC	+ UNet residual correction	+481k	—

Table 1: Comparison of transient surrogates on the test set. Numeric entries to be filled.

4 CONCLUSION

This work presents a complete hierarchy of neural surrogate architectures for PDE emulation, progressing from classical POD regression to sophisticated non-linear latent-space generative models. The preliminary study on the stationary convection-diffusion problem demonstrates the effectiveness of the VAE two-stage paradigm and validates the infrastructure for data generation, normalisation, and inference.

For the more challenging transient problem, the Tucker POD and SVD-in-Laplace approaches establish competitive baselines under a linear compression assumption. The frequency-conditioned `LaplaceAE` overcomes this limitation by learning an adaptive, non-linear latent space shared across all spectral frequencies, achieving better reconstruction at comparable latent dimensionality. The subsequent per-frequency MLP surrogates, refined through end-to-end fine-tuning, produce accurate predictions of the full spatio-temporal field.

Finally, the lightweight UNet residual corrector demonstrates that a post-processing step of modest capacity (481k parameters) can substantially reduce oscillatory artefacts without retraining the base surrogate. The resulting `CorrectedPipeline` combines the best of both worlds: a structured spectral representation with non-linear latent space learning and a data-driven correction stage.

Future directions include incorporating physics-informed constraints into the latent space, extending the framework to varying geometries, and applying the residual corrector to other reduced-order surrogate families beyond the Laplace representation.

REFERENCES

[1] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. *FiLM: Visual Reasoning with a General Conditioning Layer*. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[2] A. Ammar, M. Ben Saada, E. Cueto, and F. Chinesta. Casting hybrid twin: physics-based reduced order models enriched with data-driven models enabling the highest accuracy in real-time. *International Journal of Material Forming*, 17(16), 2024. doi:10.1007/s12289-024-01812-4, hal-05056779.